الرياضيات المتقطعة

في تخصصات الحوسبة وعصر الذكاء الاصطناعي

COMP233 - Discrete Mathematics

لماذا ندرس هذا المساق؟ وكيف يساعدنا في عصر AI و Vibe Coding؟

الذكاء الاصطناعي يستطيع كتابة الكود، والرياضيات المتقطعة تساعدك على معرفة متى تثق بهذا الكود ومتى ترفضه.

ملخص تعليمي للطلاب - الفصل الأول 2026/2027

1. لماذا هذا المساق مهم؟

الرياضيات المتقطعة ليست مجموعة قوانين منفصلة عن تخصصات الحوسبة. هي تدريب على التفكير المنطقي، بناء الحجج والبراهين، وتحويل المشكلات إلى نماذج رياضية واضحة يمكن تحليلها واختبارها.

يركز مساق COMP233 على المنطق، البراهين، المجموعات، الدوال، العلاقات، العد، الرسوم البيانية والأشجار، مع ربط هذه المفاهيم بتطبيقات الحاسوب والبرمجة.

> الفكرة المركزية: لا نريد من الطالب أن يحفظ تعريفًا؛ نريد منه أن يستخدم التعريف كأداة لاتخاذ قرار، كشف خطأ، أو تبرير حل.

2. أهمية المساق لكل تخصص

علوم الحاسوب - Computer Science

هذا المساق من أهم الأسس الرياضية لعلوم الحاسوب، لأن كثيرًا من الأشياء التي نتعامل معها في البرمجة متقطعة: أعداد صحيحة، قيم Boolean، حالات خوارزمية، هياكل بيانات، عقد وروابط.

- Logic لفهم الشروط في البرامج مثل if و and و or و not.
- Proofs لفهم لماذا الخوارزمية صحيحة، وليس فقط لأنها نجحت في بعض الأمثلة.
- Induction لفهم loops و recursion والاستدلال على السلوك المتكرر.
- Sets and Relations لفهم هياكل البيانات وقواعد البيانات.
- Counting لتحليل عدد الحالات الممكنة وحجم فضاء الحل.
- Graphs and Trees لتمثيل الشبكات، البحث، الأشجار والمسارات.

> الرياضيات المتقطعة هي اللغة الرياضية التي تُبنى عليها كثير من مفاهيم علوم الحاسوب.

هندسة الحاسوب - Computer Engineering

بالنسبة لطالب هندسة الحاسوب، تظهر أهمية المساق في الانتقال بين الرياضيات والعتاد Hardware، خصوصًا عندما تتحول التعبيرات المنطقية إلى دوائر رقمية حقيقية.

- Propositional Logic أساس لفهم الدوائر المنطقية.
- AND و OR و NOT تتحول إلى Logic Gates.
- Boolean Algebra تساعد على تبسيط الدوائر الرقمية.
- Binary representation مرتبط مباشرة بتمثيل البيانات داخل الحاسوب.
- Functions و Relations تساعد في توصيف الأنظمة والحالات والانتقالات.
- Graphs تفيد في نمذجة الشبكات والأنظمة المترابطة.

$$(p \land q) \lor \neg r$$

هذا التعبير ليس مجرد رمز رياضي؛ يمكن أن يمثل شرطًا داخل نظام رقمي أو دائرة Hardware.

Artificial Intelligence - الذكاء الاصطناعي

في الذكاء الاصطناعي نحتاج إلى تمثيل المعرفة والاستنتاج منها، ولذلك تظهر أهمية المنطق، العلاقات، الأشجار، والعد بشكل مباشر.

- Propositional Logic لتمثيل حقائق وقواعد بسيطة.
- First-Order Logic لتمثيل الخصائص والعلاقات باستخدام predicates و quantifiers.
- Rules of Inference لاستخلاص معرفة جديدة من معلومات موجودة.
- Relations لتمثيل العلاقات بين الكيانات.
- Graphs and Trees لتمثيل البحث و Knowledge Graphs.
- Counting لفهم حجم Search Space.

$$\text{Human(x)} \rightarrow \text{Mortal(x)} \quad + \quad \text{Human(Socrates)} \quad \Rightarrow \quad \text{Mortal(Socrates)}$$

Cybersecurity - الأمن السيبراني

في الأمن السيبراني، الرياضيات المتقطعة تساعد في صياغة سياسات الوصول، تحليل الاحتمالات، فهم مبادئ التشفير، ونمذجة الشبكات ومسارات الهجوم.

- Logic لكتابة وتحليل Access-Control Policies.
- Number Theory أساس مهم لفهم كثير من أفكار التشفير.
- Divisibility و mod في الحسابات والخوارزميات الأمنية.
- Counting لحساب عدد كلمات المرور أو المفاتيح الممكنة.
- Probability لتحليل المخاطر والاحتمالات.
- Graphs لتمثيل الشبكات ومسارات الوصول والهجوم.

Password space = 62^8

هنا تتحول عبارة مثل "هل فضاء كلمات المرور كبير؟" إلى سؤال يمكن قياسه رياضيًا.

3. ما دور الرياضيات المتقطعة في عصر AI و Vibe Coding؟

في البرمجة التقليدية كان السؤال الأساسي: كيف أكتب الكود؟ أما مع أدوات الذكاء الاصطناعي، فأصبح السؤال الأهم: كيف أعرف أن الكود الذي ولّده AI صحيح أصلًا؟

قد يستطيع AI أن يكتب الكود بسرعة، لكن الإنسان ما زال مسؤولًا عن تحديد المطلوب بدقة، اكتشاف التناقضات، تقييم الحل، والتحقق من صحة السلوك في الحالات المختلفة.

1) Logic: كيف تعطي AI طلبًا دقيقًا؟

كلما كان المطلوب غامضًا، زادت احتمالية أن يعطيك AI برنامجًا يبدو صحيحًا لكنه لا يحقق ما تريد. فهم AND و OR و NOT و implication يساعدك على صياغة prompts ومتطلبات وسياسات أكثر دقة.

(Admin ∨ Permission) ∧ ¬Suspended

مثال: اسمح للمستخدم بالدخول إذا كان Admin أو لديه Permission، وبشرط ألا يكون الحساب Suspended.

2) Proofs: لا تسأل فقط "هل يعمل؟" بل "لماذا يعمل؟"

قد ينجح الحل الذي ولّده AI على عشرات الأمثلة ثم يفشل في حالة لم تختبرها. التفكير البرهاني يدفعك إلى سؤال: ما الافتراضات؟ ماذا يجب أن نثبت؟ هل توجد حالة مضادة؟ وهل الحل صحيح لكل input أم لبعض الأمثلة فقط؟

> في عصر AI يتحول دور المبرمج تدريجيًا من **Code Writer** فقط إلى **Code Reviewer + Reasoner**.

3) Sets, Relations, Functions: افهم النظام قبل أن تقبل الكود

عندما تطلب من AI بناء Database أو Recommendation System أو Social Network أو Access-Control System أو API، فأنت تتعامل فعليًا مع مجموعات وعلاقات ودوال. إذا كان النموذج المفاهيمي خاطئًا، فقد يكون الكود مرتبًا لكنه يبني النظام الخطأ.

- Users = مجموعة.
- Products = مجموعة.

- Purchased(user, product) = علاقة.
- Email(user) = دالة.

4) **Induction**: فهم **loops** و **recursion** بدل الثقة العمياء

يمكن لـ AI توليد recursive function خلال ثوانٍ، لكن يجب أن تسأل: هل ستتوقف؟ هل تغطي جميع الحالات؟ هل النتيجة صحيحة لكل n؟ وما الخاصية التي تبقى صحيحة بعد كل iteration؟

5) **Counting**: هل الحل الذي اقترحه **AI** عملي؟

قد يقترح AI تجربة جميع الحالات الممكنة. قبل قبول هذا الحل، احسب حجم فضاء البحث. العد يساعدك على اكتشاف أن brute force قد يكون غير عملي حتى لو كان الكود نفسه صحيحًا.

For 12 characters from 62 symbols: 62^12 possibilities

6) **Graphs and Trees**: بنية كثير من أنظمة **AI**

Search Trees و Knowledge Graphs و Dependency Graphs و Agent Workflows و Computer Networks كلها أمثلة يمكن فهمها باستخدام الرسوم البيانية والأشجار.

المفارقة المهمة في عصر **Vibe Coding**

قد يسأل الطالب: إذا كان AI سيكتب الكود عني، لماذا أتعلم الرياضيات المتقطعة؟ الجواب أن كتابة الكود أصبحت أسهل وأسرع، ولذلك أصبحت قدرتك على التفكير والتحقق أهم من قدرتك على كتابة كل سطر بنفسك.

> في الماضي كان المبرمج المتميز هو الذي يستطيع كتابة الكود. في عصر **AI**، المبرمج المتميز هو الذي يعرف ماذا يطلب، وكيف يصيغ المشكلة، وكيف يكتشف أن جواب **AI** خطأ، وكيف يبرر أن الحل صحيح.

أسئلة يجب أن تبقى مع الطالب عند استخدام **AI**

- هل المشكلة صيغت بشكل صحيح؟
- هل الشروط منطقية ومتسقة؟
- هل الحل صحيح لكل الحالات؟
- هل توجد حالة مضادة؟

- هل الخوارزمية قابلة للتوسع؟
- هل هناك تناقض أو حالة ناقصة في المتطلبات؟
- هل النظام آمن؟

> **.AI writes code; Discrete Math helps you trust - or reject - it**

4. الخيط التربوي الذي يربط فصول المساق

لا نريد أن يشعر الطالب أن الفصول موضوعات منفصلة. يمكن تقديم المساق كرحلة واحدة تبدأ بالتفكير المنطقي وتنتهي بنمذجة الأنظمة والشبكات.

Logic → Proof → Structures → Relationships → Counting → Networks

- كيف أفكر بشكل صحيح؟
- كيف أثبت أن ما أقوله صحيح؟
- كيف أمثل البيانات؟
- كيف أمثل العلاقات بين البيانات؟
- كم احتمالًا يوجد؟
- كيف ترتبط الأشياء معًا في شبكة؟

> هدفنا في **Discrete Mathematics** ليس أن تصبح أسرع من **AI** في كتابة الكود، بل أن تصبح أفضل في فهم المشكلة، صياغة شروطها، واكتشاف متى يكون الحل غير صحيح.

5. مرتكزات هذا العرض

- الخطة الرسمية لمساق COMP233 - Discrete Mathematics، الفصل الأول 2026/2027.
- Susanna S. Epp, Discrete Mathematics with Applications, 4th Edition.
- Birzeit University / Jarrar Lecture Notes في المنطق، نظرية الأعداد، الاستقراء، المجموعات، الدوال، العلاقات، والعد.
- الأمثلة المرتبطة بالتخصصات وAI وVibe Coding صيغت هنا كأمثلة تعليمية لتقريب المفاهيم للطلاب.

6. التأمل التربوي لكل فصل مع مثال واقعي لكل تخصص

السؤال الذي يقود هذا الجدول: ما الفكرة التي نريد أن تبقى في عقل الطالب بعد أن ينسى معظم القوانين؟ وكيف تغيّر هذه الفكرة طريقة تعامله مع مشكلات تخصصه؟

الفصل	التأمل التربوي: ماذا نريد أن يتغير في طريقة تفكير الطالب؟	علوم الحاسوب CS	هندسة الحاسوب CE	الذكاء الاصطناعي AI	الأمن السيبراني Cybersecurity
Ch. 2 Propositional Logic	لا تقبل جملة غامضة قبل أن تحدد معناها المنطقي. يتعلم الطالب تحويل اللغة الطبيعية إلى شروط دقيقة، والتمييز بين ما يبدو صحيحًا وما هو صحيح منطقيًا.	مراجعة شرط برمجي مثل: if (admin || owner) && !blocked. هل يعبّر فعلًا عن المطلوب؟	تصميم دائرة Alarm تعمل عند تحقق شرط معين باستخدام AND / OR / NOT gates.	تمثيل قواعد بسيطة مثل: إذا تحقق A وB فاستنتج C، ثم فحص اتساق القواعد.	صياغة سياسة دخول: يسمح بالدخول إذا كان المستخدم موظفًا ولديه صلاحية وليس حسابه موقوفًا.
Ch. 3 First-Order Logic	العالم ليس مجرد True/False؛ توجد أشياء وخصائص وعلاقات بينها. يتعلم الطالب الانتقال من الجمل العامة إلى نماذج Formal باستخدام predicates وquantifiers.	التعبير عن متطلبات نظام: كل مستخدم لديه حساب واحد على الأقل، ثم البحث عن الحالات المخالفة.	توصيف مكونات Hardware: كل Processor مرتبط بذاكرة، أو يوجد Sensor يراقب كل وحدة.	Knowledge Representation باستخدام قواعد مثل Student(x) → Person(x) وعلاقات بين الكيانات.	التعبير عن سياسة: كل ملف سري يجب أن يكون له مستخدم مخوّل بالوصول إليه.

الفصل	التأمل التربوي: ماذا نريد أن يتغير في طريقة تفكير الطالب؟	علوم الحاسوب CS	هندسة الحاسوب CE	الذكاء الاصطناعي AI	الأمن السيبراني Cybersecurity
Ch. 4 Number Theory & Proof	لا يكفي أن تعمل الفكرة في أمثلة؛ يجب أن أعرف لماذا تعمل دائمًا. ينتقل الطالب من التجريب إلى البرهان والتفكير العام.	إثبات أن خوارزمية تعتمد على parity أو divisibility تعطي نتيجة صحيحة لكل input.	استخدام mod في counters دورية، أو تقسيم clock cycles العناوين.	استخدام modular arithmetic أو parity في خوارزميات preprocessing أو تقسيم البيانات.	تطبيق mod وdivisibility في مبادئ التشفير وhashing.
Ch. 5 Sequences & Induction	إذا كان النظام يتكرر، ابحث عن النمط ثم أثبت أنه يستمر. الاستقراء يعلّم base case ثم الانتقال من k إلى k+1 بدل حفظ قالب.	إثبات صحة recursive function أو loop لكل n، وليس تجربته على عدد قليل من المدخلات.	تحليل نظام يتكرر كل clock cycle أو سلسلة من حالات Hardware.	فهم recursive search أو خوارزمية تبني الحل خطوة بعد خطوة.	إثبات أن إجراء أمني recursive أو عملية تحقق متعددة المراحل تحافظ على invariant معين.
Ch. 6 Set Theory & Boolean Algebra	قبل أن تبرمج البيانات، حدد من ينتمي إلى ماذا وكيف تتقاطع الفئات. المجموعات تساعد على بناء نموذج واضح قبل كتابة الكود.	تمثيل Users وCourses وPermissions كمجموعات، واستخدام intersection لمعرفة العناصر المشتركة.	استخدام Boolean Algebra لتبسيط دائرة منطقية وتقليل عدد gates.	تقسيم Dataset إلى مجموعات مثل training/test أو تقاطع مستخدمين يحققون عدة خصائص.	تعريف مجموعات المستخدمين المسموح لهم بالدخول ثم استخدام union / intersection / difference لبناء Access Control.

الفصل	التأمل التربوي: ماذا نريد أن يتغير في طريقة تفكير الطالب؟	علوم الحاسوب CS	هندسة الحاسوب CE	الذكاء الاصطناعي AI	الأمن السيبراني Cybersecurity
Ch. 7 Functions	كل input يجب أن تكون له نتيجة محددة وفق قاعدة واضحة. يتعلم الطالب التفكير في domain وcodomain وrange، لا في الصيغة فقط.	التفكير في API كدالة: Request معين ← Response محدد، وما هو المجال الصحيح للمدخلات؟	دالة تحول Sensor Input إلى Output أو mapping بين addresses ومواقع memory.	نموذج ML كدالة تقريبًا: Features ← Prediction، ثم فحص domain والoutputs الممكنة.	Hash function: Message ← Hash Value، ثم مناقشة لماذا ليست بالضرورة one-to-one.
Ch. 8 Relations	ليست كل علاقة دالة؛ أحيانًا المهم هو شبكة العلاقات نفسها. يتعلم الطالب تحليل reflexivity وsymmetry وtransitivity.	قاعدة بيانات: EnrolledIn(Student, Course) أو Follows(User, User) وتحليلها كrelation.	علاقة اتصال بين Devices: أي الأجهزة تتصل بأي أجهزة أخرى؟	مثل Knowledge Graph Person - WorksAt - Company أو Drug - Treats - Disease.	علاقة "يمكنه الوصول إلى": هل الانتقال من A إلى B ثم B إلى C يسمح بالوصول من A إلى C؟ قد يكشف ذلك مسار هجوم.
Ch. 9 Counting Theory	قبل أن تكتب برنامجًا يجرب كل الاحتمالات، احسب عددها. حجم فضاء الحل يحدد إن كان الحل عمليًا أصلًا.	حساب عدد حالات الاختبار أو configurations قبل اختيار brute-force algorithm.	حساب عدد configurations الممكنة لمجموعة switches أو components.	حساب حجم search space: إذا كان لكل خطوة b خيارات وعمق البحث d، فإن عدد المسارات قد ينمو بسرعة كبيرة.	حساب password/key space؛ مثلًا كلمة مرور من 12 رمزًا من 62 خيارًا تعطي 62^12 احتمالًا.

الفصل	التأمل التربوي: ماذا نريد أن يتغير في طريقة تفكير الطالب؟	علوم الحاسوب CS	هندسة الحاسوب CE	الذكاء الاصطناعي AI	الأمن السيبراني Cybersecurity
Ch. 10 **Graphs & Trees** (إذا سمح الوقت)	بعض المشكلات تصبح أسهل عندما نحولها إلى nodes وedges. الهدف هو إدراك أن الطرق والاتصالات والdependencies والبحث يمكن تمثيلها بالبنية نفسها.	Graph لتمثيل dependencies بين packages أو لإيجاد shortest path في network.	رسم Network Topology بين routers أو components وتحليل الوصول بين العقد.	Search Tree أو Decision Tree أو Knowledge Graph؛ وهي بنى أساسية في كثير من خوارزميات AI.	Attack Graph يوضح كيف قد ينتقل المهاجم من جهاز مخترق إلى أجهزة أخرى في الشبكة.

الخلاصة للطلاب:

CS: صحة الخوارزمية | **CE**: منطق النظام | **AI**: تمثيل واستدلال | **Cybersecurity**: شروط واحتمالات وأمان